

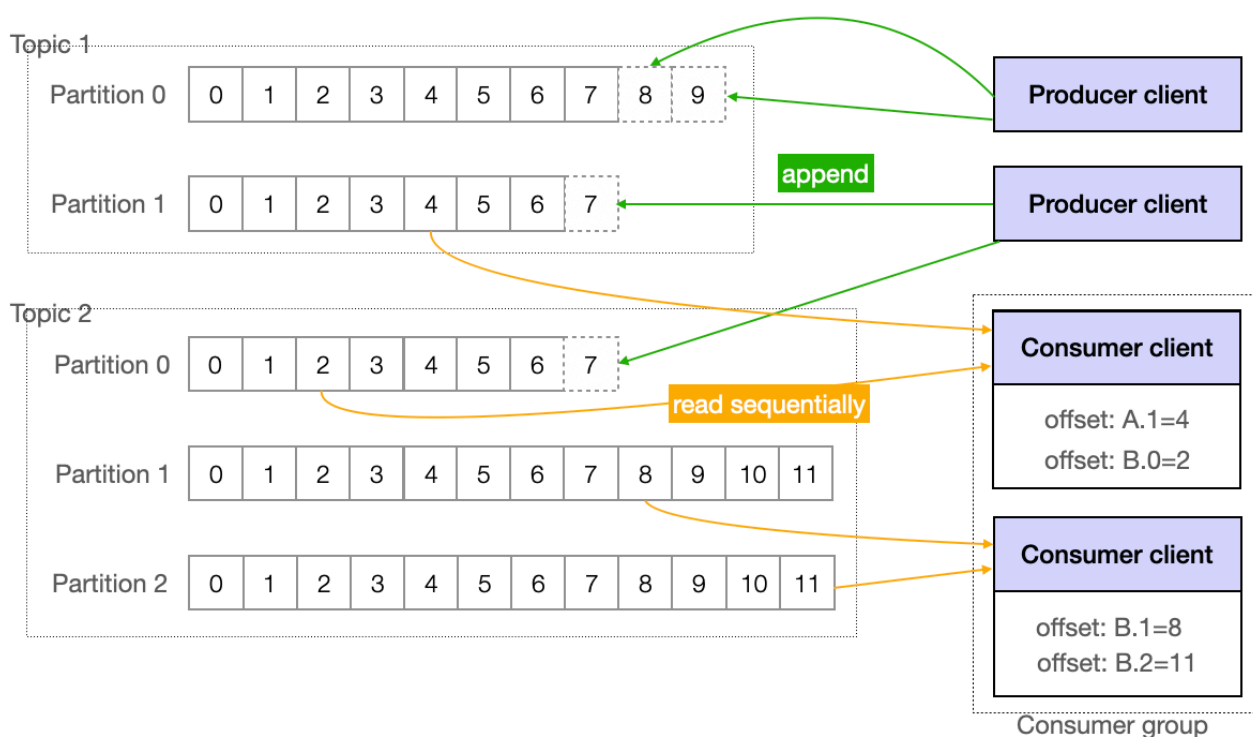
Log-based message queues

 systemdesignschool.io/fundamentals/log-based-message-queues



Message queues like Redis Queue use in-memory processing and have fast message delivery. However, they don't support long-term storage, and once messages are processed, they are deleted. Another type of message queues is log-based message queues, which provide a more flexible and scalable messaging infrastructure. Additionally, because log-based message queues store messages on disk, **one message can be consumed by multiple consumers**. This enables design patterns where one event is processed by multiple services. For example, a hotel booking service could send booking information to a log-based message queue, and then different services, such as order processing, customer notification, and data analysis services, could independently consume this message to complete their respective tasks. This approach not only ensures data consistency and reliability but also enhances the overall efficiency and flexibility of the system.

Components of a Log-based Message Queue



Components of a log-based Message Queue

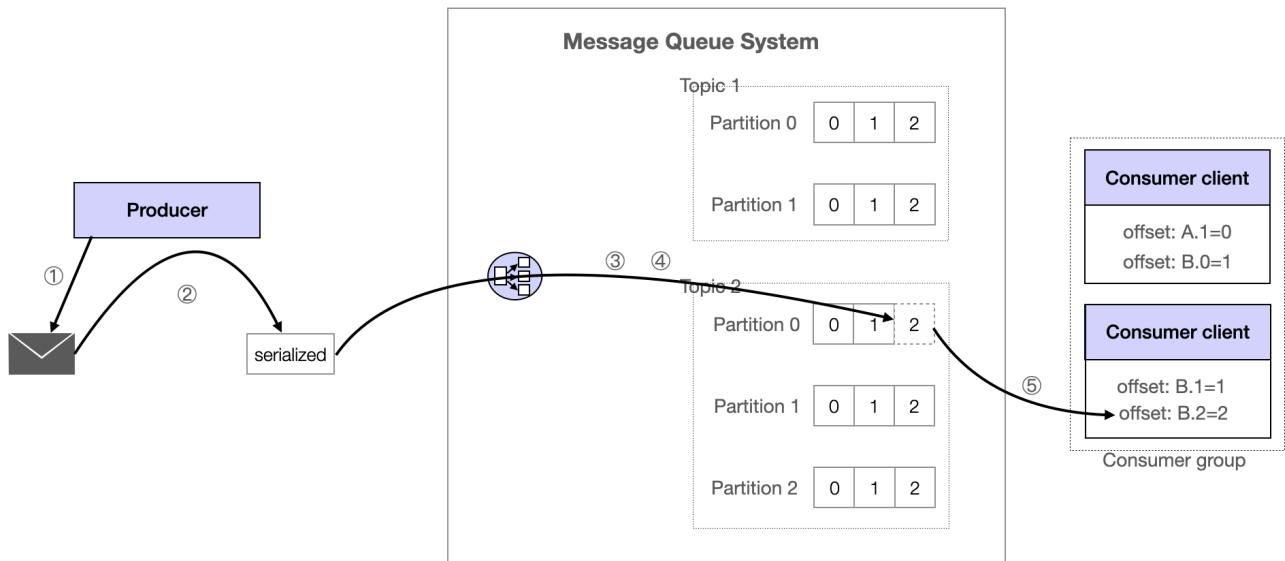
1. **Log Structure:** A log is an ordered, append-only data structure that consists of a sequence of records called messages. Each message in the log has a unique, monotonically increasing sequence number called an **offset**. Logs can be partitioned and distributed across multiple nodes in a cluster to enable horizontal scaling and parallel processing.
2. **Producers:** Producers are components or services that generate and send messages to the message queue. They write messages to the log by appending them to the end of the log. Each appended message gets a new offset, ensuring a total order within the partition.
3. **Partitions:** Partitions divide the log into smaller, more manageable segments. Each partition is an ordered, append-only sequence of messages. Partitions can be distributed across multiple nodes in a cluster, providing parallelism and increased throughput. Partitioning also enables load balancing and fault tolerance by allowing replicas to be created for each partition on different nodes.
4. **Consumers:** Consumers are components or services that read and process messages from the log. Each consumer maintains its own offset, which represents its position in the log. Consumers read messages sequentially, starting from their current offset, and update their offset as they process messages. This approach allows multiple consumers to read from the same partition independently, without affecting each other.

5. **Consumer Groups:** Consumers can be organized into consumer groups to enable parallel processing and load balancing. Each consumer group processes messages from a distinct set of partitions. When a consumer fails or is added to the group, the partitions are reassigned among the available consumers in the group, ensuring fault tolerance and scalability.

Additionally, partitions can be replicated across many nodes, similar to what was described in the replication section.

When a new message arrives in a log-based message queue, several steps occur to ensure proper handling and storage:

1. **Message Production:** A producer component or service generates a new message and sends it to the message queue. The producer may include key-value pairs (metadata), a topic or partition identifier, and the actual message payload.
2. **Message Serialization:** The producer may serialize the message (convert it into a byte stream) to ensure efficient storage and transmission. Common serialization formats include JSON, Avro, or Protocol Buffers.
3. **Partition Selection:** If the producer has not explicitly specified a partition, the message queue system may select a partition for the message based on a partitioning strategy. Common strategies include round-robin or using a partition key (e.g., a hash of the message key) to ensure an even distribution of messages across partitions.
4. **Message Appending:** The message is appended to the end of the selected partition in the log. This operation is usually very fast because it involves a sequential write to disk. The message is assigned a unique, monotonically increasing offset within the partition, ensuring a total order of messages.
5. **Message Consumption:** Consumers that are subscribed to the topic or partition will eventually process the new message. They read messages sequentially from their current offset and update their offset as they process messages. If multiple consumers belong to the same consumer group, they share the processing load by reading from different partitions.



Note that in a typical setup, one partition can only be read by one consumer at a time. If a single message is slow to process, it holds up the processing of subsequent messages in that partition and therefore degrades performance. This is why in the message queue overview section we suggest using a traditional AMQP-style queue if messages could be expensive to process.

Also note the similarity of the message log to the replication log in leader-based replication we discussed in an earlier chapter. In both cases, the consumer uses an offset to record the position it has consumed and resume processing if it disconnects.

Use cases of log-based message queues

There are many use cases, most notably:

- **Service-to-service communication:** Just like in the AMQP-style message queues, log-based message queues can be used as a link between services.
- **Event sourcing:** Log-based message queues can be used to store a sequence of events that represent changes in application state. This allows applications to reconstruct the current state by replaying the events in order.
- **Stream processing:** Log-based message queues can be used to process and analyze real-time data streams. For instance, you can use Kafka Streams or Apache Flink to process and aggregate data from multiple sources, perform windowed computations, and trigger actions based on the processed data.

Popular Log-based message queues

Log-based Message Queue	Developer/Provider	Key Features
Apache Kafka	Apache Software Foundation	Highly scalable, distributed, fault-tolerant, publish-subscribe messaging system
Apache Pulsar	Yahoo (Apache Foundation)	Scalable, low-latency, distributed messaging platform, multi-tenancy support
Amazon Kinesis	Amazon Web Services (AWS)	Real-time data streaming and processing, managed service, integration with other AWS services
Google Cloud Pub/Sub	Google Cloud Platform	Real-time messaging between independent applications, supports streaming and batch processing
NATS Streaming	http://nats.io/	High-performance, distributed, low-latency messaging, at-most-once or at-least-once message delivery
Apache Flink	Apache Software Foundation	Distributed stream processing framework, works with other log-based message queues like Kafka
Apache Samza	Apache Software Foundation	Distributed stream processing framework, used with Kafka, stateful and fault-tolerant stream processing

Let's briefly introduce Kafka as it's the one that's mostly used.